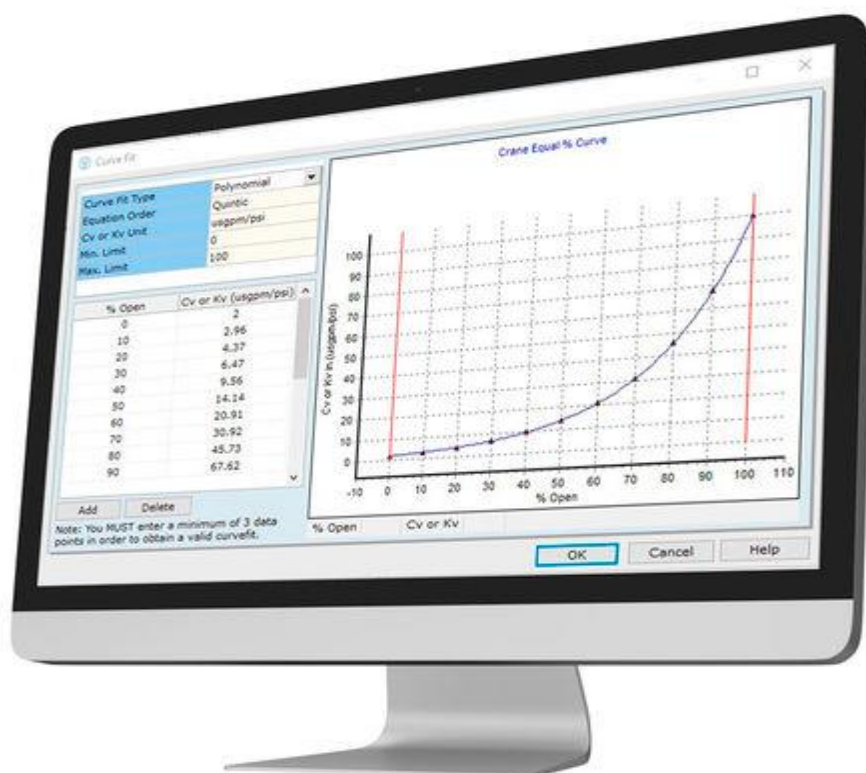


Département de Physique

TRAVAUX PRATIQUES

DE LA DYNAMIQUE DES FLUIDES

(SIMULATION)



**Génie Mécanique- Productique et
Thermique (2^{ème} Année)**

Responsable : A. OUSEGUI
Année universitaire : 2025 – 2026

Avant-propos

Ce TP a pour objectif d'initier les étudiants, à un autre aspect de la résolution des problèmes de mécanique des fluides, à savoir : "la modélisation numérique". En effet certains problèmes n'admettent pas de solution analytique, et le recours à l'analyse numérique s'avérerait incontournable. On insistera sur les différents termes de l'équation régissant le phénomène considéré, et la façon de la résoudre.

Les sujets proposés demeurent en symbiose avec le contenu du cours de la dynamique des fluides. Ainsi, on étudiera les problématiques liées aux écoulement visqueux sous pression ou en conduites fermées. La traînée, la portance et les problèmes de couche limite, Et par la suite les équations des écoulements compressibles.

Le logiciel utilisé serait Matlab. Du fait une introduction du logiciel accompagnée d'un petit rappel des commandes usuelles feront office du début de la 1^{ère} séance.

A la fin des séances de ce TP, l'étudiant sera apte (on l'espère) à développer des algorithmes de calcul d'une part et d'autre part, à utiliser le logiciel d'une façon optimale.

Pr. Abdellah OUSEGUI

SOMMAIRE

INTRODUCTION AU MATLAB	1
PERTES DE CHARGE	14
AÉRODYNAMIQUE - ÉTUDE COMPARATIVE DES PROFILS NACA 2412 ET NACA 0012	18
ÉTUDE DES ÉCOULEMENTS COMPRESSIBLES DANS UNE TUYERE DE LAVAL	23

INTRODUCTION AU MATLAB

Ce chapitre est largement inspiré du document "Premiers pas en Matlab" de l'auteur Pr. Florent Krzakala ([matlab.pdf](#))

Introduction : qu'est-ce que Matlab ?

Le logiciel Matlab constitue un système interactif et convivial de calcul numérique et de visualisation graphique. Destiné aux ingénieurs, aux techniciens et aux scientifiques, c'est un outil très utilisé, dans les universités comme dans le monde industriel, qui intègre des centaines de fonctions mathématiques et d'analyse numérique (calcul matriciel le MAT de Matlab, traitement de signal, traitement d'images, visualisations graphiques, etc.). Notons que d'autres logiciels compatibles avec Matlab existent ; citons par exemple OCTAVE sous Linux ou encore SCILAB, développé en France par l'INRIA, qui sont tous deux très performants et de plus gratuits (téléchargement libre).

I. Démarrer et quitter Matlab

Pour lancer Matlab, tapez

```
>> matlab
```

cliquez simplement sur l'icône dans un environnement. Pour quitter, tapez

```
>> quit
```

ou

```
>> exit
```

Matlab fonctionne à l'aide de commandes qu'il faut apprendre à utiliser. Vous pouvez à n'importe quel moment taper

```
>> help <command>
```

pour savoir comment on utilise telle ou telle commande (par exemple essayez **help exit**). Le module d'aide est aussi assez efficace ; on peut par exemple une recherche de mots clefs en tapant

```
>> lookfor sinus
```

on trouvera que la commande *sin* correspond, sans surprise à la fonction sinus.

On peut enfin assister à une démonstration de Matlab en tapant

```
>> demo matlab
```

II. Les variables dans Matlab, premiers calculs

Dans un premier temps, Matlab peut bien sûr s'utiliser comme une vulgaire calculatrice. Ainsi

```
>> 1+2
```

donnera 3,

```
>> 4^2
```

retourne bien 16 et

```
>> (((1+2)^3)*4)/2
```

donne bien 54, puisque Matlab respecte les règles usuelles des mathématiques. Il est aussi possible, comme dans tout langage de programmation, de définir des variables pour stocker un résultat ; ainsi après

```
>> a=1+2
```

```
>> b=2+3
```

```
>> a+b
```

retourne bien 8. Si l'on veut éviter que Matlab affiche les résultats intermédiaires, on ajoute simplement un ";" à la fin de la commande. On vérifiera que

```
>> a=1+2 ;
```

```
>> b=2+3 ;
```

```
>> a+b
```

effectue les mêmes opérations que précédemment, mais sans afficher les résultats intermédiaires. Enfin, les calculs avec les variables complexes sont parfaitement possibles, en utilisant i ou j (avec bien évidemment $i^2 = j^2 = -1$) :

```
>> a=1+2i ;
```

```
>> b=2+3i ;
```

```
>> a*b
```

donne la réponse attendue à savoir **ans = -4 + 7i**. On peut ainsi vérifier que

```
>> exp(i * pi)+1
```

est bien nul (aux approximations numériques près), et que

```
>> sqrt(-1)
```

retourne $-i$. Matlab étant très tolérant, il est possible d'appeler une variable i (ou bien j), mais dans ce cas, on ne pourra plus utiliser par la suite i comme étant le générateur des nombres complexes. Il est donc utile de savoir quelles sont les variables que nous avons utilisées depuis le début de la session, et pour cela il suffit d'écrire

```
>> who
```

et pour en savoir plus sur elles

```
>> whos
```

Enfin, pour les détruire, on utilise

```
>> clear a
```

ou encore pour toutes les variables

```
>> clear all
```

Certaines variables très utiles sont déjà définies, comme

```
>> pi
```

3.1416

Ou

```
>> ans
```

qui retourne la dernière réponse donnée par Matlab. Il est aussi utile de savoir qu'en tapant sur ↑ on peut rappeler les commandes précédentes (la flèche ↓ permettant de revenir) cela dispense de taper plusieurs fois des commandes identiques.

III. L'opérateur colon “:”

Avant d'aller plus loin, il est nécessaire de se familiariser avec une notation que l'on retrouve partout en Matlab, celle de l'opérateur “:” (en anglais **colon**). En Matlab, quand l'on écrit “1 : 10” par exemple, cela signifie “tous les nombres de 1 à 10” (c.a.d. 1 2 3 4 5 6 7 8 9 10), et “1 : 2 : 10” signifie “tous les nombres de 1 à 10 avec un pas de 2 (et donc les nombres : 1 3 5 7 9)”, le pas étant unitaire par défaut. Cette notation doit être impérativement comprise car elle est utilisée sans arrêt. On s'amusera par exemple à écrire

```
>> 1 :10
```

```
>> 10 :-1 :1
```

```
>> 0 :pi/4 :pi
```

et à observer le résultat.

IV. Les vecteurs en Matlab

Nous n'avons vu ici que des variables scalaires (i.e. des nombres) mais Matlab permet d'utiliser aussi bien des vecteurs ou des matrices. Commençons par les vecteurs ; Pour définir un vecteur v , par exemple $v = (11 \ 22 \ 33 \ 44 \ 55)$, on utilise

```
>> v=[11 22 33 44 55]
```

La commande

```
>> v'
```

donne ensuite la transposé ce vecteur, c'est à dire un vecteur colonne. On peut aussi écrire

```
>> z=[11 ; 22 ; 33 ; 44 ; 55 ;]
```

pour obtenir directement un vecteur colonne z , mais il est bien plus simple d'écrire

```
>> z=v'
```

ou encore

```
>> z=[11 22 33 44 55]'
```

pour obtenir

$$z = \begin{pmatrix} 11 \\ 22 \\ 33 \\ 44 \\ 55 \end{pmatrix}$$

Il est aussi possible d'utiliser ces notations avec notre opérateur ":" pour définir un nouveau vecteur. Par exemple si l'on écrit [1 : 1 : 5], on définit un vecteur allant de 1 à 5 avec une incrémentation unité. Ainsi

```
>> w=[1 :1 :5]
```

ou encore

```
>> w=[1 :5]
```

nous permettent de définir le vecteur

$$w = (1 \ 2 \ 3 \ 4 \ 5)$$

Les mêmes opérations sont bien sûr possibles sur les vecteurs colonnes, par exemple

```
>> vec=[1 :2 :10]'
```

définit le vecteur

$$vec = \begin{pmatrix} 1 \\ 3 \\ 5 \\ 7 \\ 9 \end{pmatrix}$$

Pour accéder à une valeur particulière d'un vecteur, on écrira par exemple

```
>> v(2)
```

qui donne simplement accès au 2^{em} élément, c.-à-d. ici la valeur 22. Il est aussi possible d'utiliser les ":" pour sélectionner plusieurs éléments d'un vecteur ! Voici quelques exemples :

```
>> v(1 :3)
```

donne accès aux éléments qui vont de 1 à 3 (avec un pas de 1 sans plus de précision, donc les éléments 1, 2 et 3). Cette commande retourne le vecteur (11 22 33). On peut aussi écrire

```
>> v(1 :2 :5)
```

qui donne accès aux éléments qui vont de 1 à 5, mais en incrémentant avec un pas de 2. Dans ce cas, le vecteur (11 33 55) est retourné. Si l'on écrit

```
>> v
```

tout le vecteur est retourné, mais l'on obtient le même résultat par

```
>> v( :)
```

le symbole ":" sans autres précisions signifiant "tout le vecteur". Matlab permet la multiplication des vecteurs, cependant on surveillera l'ordre de ce que l'on écrit : ainsi si **vec*vec**, retourne un message d'erreur, **vec'*vec** retourne une matrice et **vec'*vec** un nombre. . .

V. Les matrices en Matlab

On vient de voir notre première matrice. Il suffit en fait d'ajouter une dimension aux vecteurs pour créer des matrices avec les mêmes notations. Ainsi

```
>> A=[1 2 3; 2 4 5; 6 7 8;]
génère la matrice :
```

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 5 \\ 6 & 7 & 8 \end{pmatrix}$$

Il est important de bien faire attention à la taille des objets que l'on définit sous peine d'obtenir des messages d'erreur. Si l'on veut créer une matrice à partir de ses colonnes, il suffit d'écrire

```
>> A=[1 2 6]' [2 4 7]' [3 5 8]'
```

Pour lire les éléments, on utilise les mêmes méthodes que précédemment. Ainsi la commande

```
>> A(1,3)
```

permet d'accéder à l'élément (1,3) de la matrice A, qui est ici 3. De même

```
>> A(1 :3,2 :3)
```

retourne les lignes 1 à 3 et les colonnes 2 à 3 :

$$\begin{pmatrix} 2 & 3 \\ 4 & 5 \\ 7 & 8 \end{pmatrix}$$

Le même résultat est obtenu par la notation

```
>> A(1 :1 :3,2 :1 :3)
```

Il est enfin souvent utile de pouvoir accéder à l'un des vecteur (ligne ou colonne) d'une matrice. Il faut alors se souvenir que l'on accède à toutes les valeurs d'un vecteur par la notation ":" et que, par conséquent, on écrira $A(:,2)$ pour obtenir la deuxième colonne de A et $A(:,3)$ pour la troisième. De même $A(1,:)$ retourne toute la première ligne.

Enfin, on souhaite fréquemment transposer une matrice ; en Matlab, encore une fois, on procède comme pour les vecteurs :

```
>> A'
```

est bien la transposé de A. Il existe un certain nombre de commandes qui génèrent automatiquement des matrices particulières :

1. **rand(p)** crée une matrice $p \times p$ dont les éléments sont aléatoires, distribués selon une loi de probabilité uniforme dans l'intervalle $[0, 1[$.
2. **randn(p)** crée une matrice $p \times p$ dont les éléments sont aléatoires, distribués selon une loi de probabilité gaussienne de moyenne nulle et de variance unité.
3. **eye(p)** crée une matrice identité $p \times p$.
4. **zeros(p)** crée une matrice de zéros $p \times p$.

5. **ones(p)** crée une matrice de uns $p \times p$.

Il est aussi possible d'utiliser ces commandes pour créer des matrices non carrées, en écrivant par exemple **eye(5,3)**. On consultera avec profit l'aide en ligne de Matlab. Il est possible, pour finir, de définir les matrices par blocs. Ainsi, pour créer une matrice

$$B = \begin{pmatrix} 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 \end{pmatrix}$$

Il suffira d'écrire :

```
>> B=[eye(3) eye(3);ones(3) zeros(3);]
```

ou bien en utilisant les colonnes

```
>> B=[[eye(3) ones(3)]' [eye(3) zeros(3)]']
```

ou bien encore

```
>> B=[[eye(3) ; ones(3)] [eye(3) ;zeros(3)]]
```

VI. Opérations matricielles avec les vecteurs et les matrices

Matlab est vraiment performant pour la manipulation de matrices, de vecteurs, ou de tableaux en général. Pour la clarté de l'exposé on notera par la suite les vecteurs avec des minuscules et les matrices par des majuscules. La multiplication de vecteurs et de matrices se fait en utilisant la notation "*". Par exemple

```
>> C=[1 3 4; 5 2 1; 3 1 2]
```

```
>> b=[3 1 2]
```

```
>> v=[1 3 4]
```

```
>> v*b'
```

```
>> v'*b
```

```
>> A*C
```

```
>> C*A
```

```
>> A*v'
```

Notez bien évidemment que cette multiplication n'est pas commutative, comme il se doit. Pour les matrices carrées, il existe aussi des fonctions spécifiques extrêmement utiles :

6. **inv(A)** calcule l'inverse de A.

7. **det(A)** calcule le déterminant de A.

8. **diag(A)** retourne la diagonale.

9. **[V,E]=eig(A)** retourne deux matrices V et E. V est une matrice dont les colonnes contiennent les vecteurs propres de A, et E est une matrice dont la diagonale contient les valeurs propres de A.

Matlab permet aussi d'effectuer des inversions de matrices, et donc de résoudre rapidement des systèmes d'équations linéaires. Par exemple, pour résoudre un système du type $A \cdot x = b$, la solution est donnée par $x = A^{-1} \cdot b$. Cela donne, en Matlab :

```
>> inv(A)*b
```

On peut aussi écrire

```
>> A \ b
```

où l'on notera le symbole \ qui signifie diviser par la gauche ou plus directement, si l'on tient à utiliser la division usuelle (la division à droite)

```
>> eye(3)/A*b
```

VII. Opérations dans les éléments des matrices/vecteurs

Si nous voulons travailler directement avec les éléments de matrices/vecteurs, par exemple multiplier les éléments de A par ceux de B , il ne s'agit plus alors d'une opération matricielle, mais bien d'une multiplication des éléments de matrices. C'est parfois utile quand on utilise un vecteur comme un tableau. Pour spécifier que l'on utilise les éléments et non pas les matrices, on écrit par exemple pour la multiplication “.” au lieu de “*”. Ainsi on aura pour la multiplication élément par élément des vecteurs

```
>> v.*b
```

ou

```
>> b.*v
```

et de même pour des matrices

```
>> A.*C
```

```
>> b./v
```

```
>> C./A
```

```
>> A.^2
```

Certaines fonctions, qui ne sont pas destinées directement aux matrices, s'appliquent directement aux éléments et n'ont donc pas besoin du point :

```
>> sin(v)
```

```
>> log(C)
```

```
>> exp(A)
```

```
>> 0.5-C
```

Enfin, les fonctions suivantes sont parfois très utiles : $[m, n] = \text{size}(A)$, retourne m , le nombre de lignes, et n le nombre de colonnes de A . $\text{min}(V)$ retourne un vecteur qui contient l'élément le plus petit de chaque colonne de V (ou le plus petit élément de V si V est un vecteur). De même $\text{min}(\text{min}(V))$ retourne l'élément le plus petit de la matrice V et $\text{max}(A)$ effectue les mêmes opérations pour les maximums.

VIII. Graphes et dessins

Il est facile d'effectuer diverses représentations graphiques avec Matlab, en 2d aussi bien qu'en 3d.

En tapant

```
>> help plot
```

on a accès à toutes les options disponibles. On pourra aussi s'aider de la demo Matlab. Pour afficher la fonction $x \sin(x)$ dans l'intervalle $[0 : 10]$, il suffit d'écrire

```
>> x=[0 :0.1 :10] ;
```

```
>> y=x.*sin(x) ;
```

```
>> plot(x,y)
```

Notons que l'on a ici besoin de la multiplication "avec le point" (.*) puisque x et y sont des vecteurs.

Enfin, on peut nommer les différents graphes de la façon suivante :

```
>> xlabel('x')
```

```
>> ylabel('y')
```

```
>> title('...')
```

On peut aussi afficher deux courbes à la fois en tapant **hold on** après le premier plot et écrire **hold off** après le dernier. Ou encore plus simplement

```
>> plot(x,sin(x),x,cos(x))
```

On peut aussi diviser l'écran en deux parties dans le même graphe

```
>> subplot(1,2,1)
```

```
>> plot(x,sin(x),'sin')
```

```
>> subplot(1,2,2)
```

```
>> plot(x,cos(x),'cos')
```

Pour créer un fichier image, on peut soit utiliser les menus déroulants avec la souris ou encore par exemple pour un fichier PostScript (.ps) taper

```
>> print -dps name.ps
```

Il est souvent utile de tracer des courbes en utilisant des axes logarithmiques. Avec Matlab, il suffira d'utiliser **loglog**, **semilogx** et **semilogy** à la place de **plot**. On pourra aussi créer des vecteurs avec des espacements logarithmiques en utilisant

```
>> y=logspace(-1,2,6)
```

pour un vecteur de 6 valeurs allant de 10^{-1} à 10^2 . Il est enfin possible de tracer des fonctions en 3d, de la même façon qu'en 2d, en utilisant la fonction **plot3**, similaire à la fonction 2d. Par exemple, pour tracer une hélice on écrira :

```
>> t = 0 :pi/50 :10*pi ;
```

```
>> plot3(sin(t),cos(t),t) ;
```

Pour tracer des surfaces en 3d dimension, il est nécessaire d'introduire une maille (en anglais *mesh*).

L'exemple suivant montre comment utiliser cette nouvelle fonction

```
>> [x,y]=meshgrid(-2 :0.2 :2,-2 :.2 :2) ;
```

```
>> z=x.*exp(-x.^2-y.^2)
```

```
>> mesh(z)
```

On pourra s’amuser à essayer les différentes commandes **mesh**, **surf**, **surfl** ou encore **pcolor** pour les plots “de contours”.

Nous avons vu comment tracer des fonctions, mais souvent nous voulons afficher des points écrits dans un fichier. Pour accéder à ces valeurs, le plus simple est d’utiliser la fonction Matlab “load”. Si par exemple nous avons un fichier data.txt avec le format suivant

```
0  1.  
1  1.1  
2  1.5  
3  1.4  
4  1.7  
... ..
```

il suffira, pour afficher la seconde colonne en fonction de la première, d’écrire

```
load data.txt ;  
x=data( :,1) ;  
y=data( :,2) ;  
plot(x,y)
```

La première commande lit le fichier et crée une matrice (avec le même nom) contenant ces données. On copie ensuite la première colonne dans le vecteur x, la seconde dans le vecteur y, et l’on affiche finalement le graphe. On procède trivialement de la même façon en 3d.

IX. Les boucles et les instructions conditionnées

L’utilisation des boucles est le premier pas dans la programmation. En Matlab, les boucles **for** et **while** sont très utilisées pour les processus itératifs. La boucle **for** est associée à une variable, et exécute un processus plusieurs fois en prenant à chaque fois une nouvelle valeur pour cette variable. Pour illustrer cela, créons d’abord un vecteur de taille 5 rempli de valeurs aléatoires

```
>> v=rand(1,5)
```

Si l’on veut soustraire à tous les éléments (sauf le premier) de ce vecteur v la première valeur (i.e. $v(1)$), on écrit

```
>> for i = 2 : 5
```

```
v(i)=v(i)-v(1)
```

```
end
```

La première ligne se comprend pour toutes les valeurs qui vont de 2 à 5, exécute les commandes suivantes, jusqu’à end. Un exemple simple d’utilisation de boucle pour la résolution d’un vrai problème est celui des équations différentielles. Supposons que l’on veuille calculer $y(x)$, qui vérifie :

$$\frac{dy}{dx} = x^2 - y^2$$

avec $y(0) = 1$. On peut discrétiser l'équation différentielle (c'est la méthode d'Euler) ce qui donne $y(x + \Delta x) = y(x) + \Delta x(x^2 - y^2)$. Cette équation nous dit qu'il est possible de calculer $y(x + \Delta x)$ si l'on connaît $y(x)$, ce qui permet une solution itérative. Notons $h = \Delta x$, il suffit ensuite de procéder comme suit pour résoudre l'équation différentielle :

```
>> h=0.1
>> x=[0:h:2];
>> y=0*x;
>> y(1)=1
for i = 1:length(x)-1
    y(i+1) = y(i) + h * (x(i)^2 - y(i)^2);
end
```

Notons ici un piège classique : Matlab numérote ses éléments de matrices/vecteurs en partant de 1. Ce qui est pour nous la composante $y(0)$ la valeur en $x = 0$ de $y(x)$ doit donc être placée à la première place du vecteur, c.-à-d. en $y(1)$ pour Matlab. De manière générale, on fera **extrêmement** attention aux indices dans la programmation.

```
>> size(x)
```

Cette dernière commande donne le résultat [1 21]. On introduit ensuite une boucle

```
>> for i=2:21
    y(i)=y(i-1)+h*(x(i-1)^2-y(i-1)^2); end
```

On peut alors afficher un graphe du résultat

```
>> plot(x,y)
```

La commande **while** fonctionne de façon similaire ; elle exécute en boucle les commandes *tant qu'une* certaine condition est satisfaite. On aurait donc pu écrire, pour un résultat identique

```
>> i=2
>> y(1)=1
>> while(i<=21)
    y(i)=y(i-1)+h*(x(i-1)^2-y(i-1)^2);
    i=i+1
end
```

Les instructions conditionnées, enfin, permettent de n'effectuer une opération *que si* une certaine condition est satisfaite. La plus simple est **if**, qui exécute des commandes **seulement si** une condition est remplie (mais à la différence de **while**, elle n'exécute ces commandes qu'une seule fois). Par exemple, si l'on veut mettre à zéro toutes les composantes supérieures à 0.2 du vecteur v , on peut écrire

```
>> for i=1:5
>> if v(i)>0.2
>> v(i)=0
>> end
>> end
```

X. Les fonctions dans Matlab

On l'a vu, Matlab a une bonne bibliothèque de fonctions mathématiques (cos,sin,exp,log,sqrt. . .) et l'on pourra par exemple vérifier que

```
>> cos(0.5)+i*sin(0.5)
      donne bien la même chose que
>> exp(i/2)
```

comme nous l'a appris Euler. on peut encore utiliser des fonctions plus exotiques (la fonction erreur **erf**, les fonctions de Bessel que l'on trouvera par **lookfor bessell** Plus important encore, on peut définir ses propres fonctions.

Pour cela, il faut créer un fichier Matlab (on dit encore un *m-file*), ce que l'on peut faire en utilisant l'éditeur prévu par matlab (fichier-¿Nouveau-¿m-file), ou soit en ouvrant votre éditeur favori. Si l'on crée par exemple un fichier **EulerCheck.m** dans lequel on écrit

```
>> function y=EulerCheck(x)
>> y=cos(x)+i * sin(x);
```

On notera le “ ; ” pour éviter que Matlab nous affiche ce qu'il calcule dans la fonction. On accédera ensuite aux valeurs de cette fonction dans la fenêtre principale de Matlab par **>> EulerCheck(0.5)**.

Si plusieurs arguments sont fournis, on doit les séparer par une virgule ; et si plusieurs sont retournés, il faut les mettre entre crochets. Par exemple (pour un fichier nommé distance.m)

```
>> function [d,dx,dy]=distance(x1,y1,x2,y2)
>> d=sqrt((x2-x1)^2 + (y2-y1)^2)
>> dx=x2-x1
>> dy=y2-y1
```

permet de retourner la distance absolue et les distance en x et y entre deux points de coordonnées (x_1, y_1) et (x_2, y_2) . En tapant dans Matlab

```
>> [dis,disx,disy]=distance(0,0,1,1);
```

On vérifiera ensuite que $dis = 1.4142$ et que $dis_x = dis_y = 1$.

XI. Fichiers exécutables : les scripts

Il arrive que l'on doive exécuter la même tâche plusieurs fois mais en changeant seulement quelques paramètres. Une bonne façon de faire cela est de créer un fichier exécutable, ou encore *script*. Continuons avec la résolution eulérienne de l'équation $y' = 1/y$. Il faut créer un fichier dont le nom porte l'extension .m. Appelons-le **SimpleEuler.m**. Ouvrez ce fichier en utilisant l'éditeur intégré dans matlab pour créer un nouveau m-file, puis tapez

```
% file : SimpleEuler.m
% To find an approximation of dy/dx=1/y
% with y(t=0)=starty
% you need first to specify h and starty
x=[0 :h :1];
```

```

y=0*x;
y(1)=starty
for i=2:max(size(y))
y(i)=y(i-1)+h/y(i-1);
end

```

Sauvegardez-le, puis tapez

```
>> help SimpleEuler
```

Ce qui retourne les commentaires après “%”. Pour l’exécuter, on a besoin de h et de $starty$

```

>> h=0.01 ;
>> starty=1 ;
>> SimpleEuler
>> plot(x,y)

```

Il est alors simple de recommencer autant de fois que l’on veut pour différentes valeurs initiales.

XII. Fichiers sous-routines : les fonctions (bis)

Il est possible de généraliser les fichiers exécutables pour créer des fonctions, ou encore des sous-routines. On l’a vu, matlab permet de créer des fonctions mathématiques mais l’on peut demander (presque) tout à une telle fonction !

Du point de vue de la programmation, la différence principale entre les fonctions et les programmes est que

1) quand on calcule des variables dans des fonctions, elles sont complètement effacées à la fin du calcul à l’exception de celles dont on désire *retourner* la valeur ;

2) qu’il est bien sûr aussi possible d’envoyer des paramètres à la fonction, ce qui fait toute sa force.

Revenons encore à notre exemple eulérien, et créons un fichier **EulerApprox.m** dans lequel on va définir une fonction du même nom (*il est important que le nom de la fonction et celui du fichier soient identiques*)

```

function [x,y]=EulerApprox(startx,h,endx,starty)
% Find the Euler approximation of dy/dx=1/y
% in the range [startx :endx]
% with the starting condition starty
% and using h as the time step
x=[startx :h :endx] ;
y=0*x ;
y(1)=starty
for i=2:max(size(y))
y(i)=y(i-1)+h/y(i-1);
end

```

On peut alors écrire dans Matlab

```

>> [x,y]=EulerApprox(0,0.01,1,1) ;
>> plot(x,y)

```

pour obtenir le même résultat que précédemment. On notera que les variables intermédiaires (c-à-d. ici la variable i par exemple) n'existent plus en dehors de la boucle. Notez la différence si l'on tape

```
>> clear all ;  
>> h=0.01 ;  
>> starty=1 ;  
>> SimpleEuler  
>> i
```

ou si l'on tape

```
>> clear all ;  
>> [x,y]=EulerApprox(0,0.01,1,1)  
>> i
```

Cela illustre le concept de variable locale : les variables utilisées dans une fonction n'existent plus en dehors de cette fonction : **c'est une notion fondamentale** que l'on retrouve dans tous les langages de programmation. Notons que dans la pratique, pour résoudre une équation différentielle, il est préférable d'utiliser plutôt les fonctions spécifiques de Matlab ; essayez **help ode23** ou **help ode45** pour apprendre à utiliser ces fonctions.

PERTES DE CHARGE

But

L'écoulement des fluides en conduites est un phénomène omniprésent dans de nombreux domaines industriels et civils : réseaux d'adduction d'eau, transport de hydrocarbures, systèmes de chauffage, circuits hydrauliques, etc. La maîtrise des pertes d'énergie lors de l'écoulement est essentielle pour le dimensionnement et l'optimisation de ces installations. Ces travaux pratiques visent à étudier les écoulements visqueux en conduites à travers trois aspects complémentaires : l'établissement des outils fondamentaux d'analyse (Manip), la résolution de problèmes complexes par méthodes itératives (Mini-projet1) et le dimensionnement optimal d'installations (Mini-projet 2).

Objectifs pédagogiques :

- Implémenter et comprendre le diagramme de Moody
- Maîtriser les méthodes de calcul des pertes de charge
- Résoudre des problèmes implicites par méthodes itératives
- Dimensionner une installation de conduites selon des contraintes techniques

I- Rappel Théorique

I-1) Nombre de Reynolds et le régime d'écoulement

$$Re = \frac{\rho V D}{\mu} = \frac{V D}{\nu}$$

ρ : la masse volumique de du fluide
(kg/m^3)

V : vitesse moyenne de l'écoulement
(m/s) ;

D diamètre de la conduite (m) ;

μ : la viscosité dynamique du fluide
($Pa \cdot s$) ;

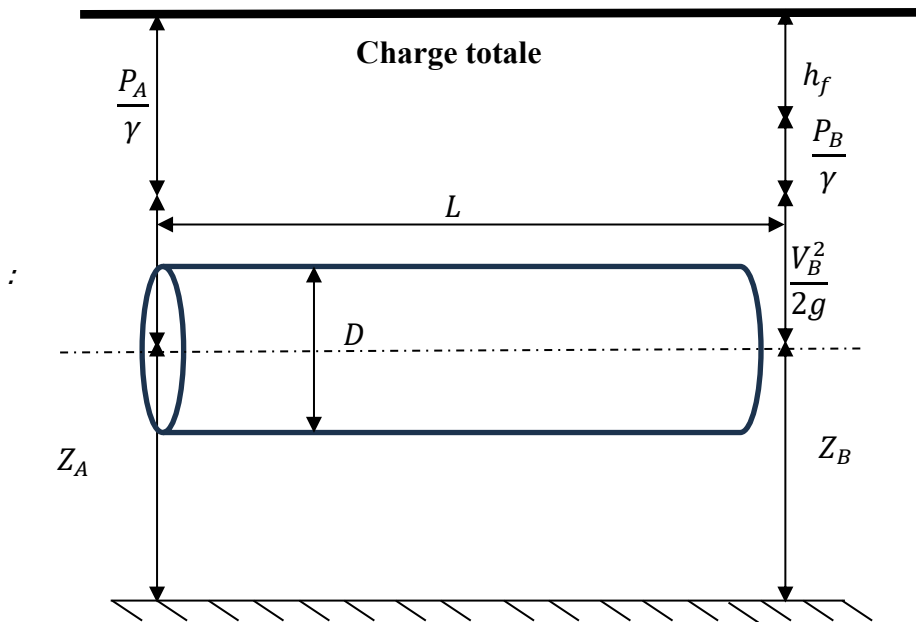
ν : la viscosité cinématique (m^2/s).

- $Re \leq 2300$ écoulement laminaire
- $2300 < Re \leq 4000$ écoulement transitoire
- $Re > 4000$ écoulement turbulent

I-2) Pertes de charge linéaires (formule de Darcy-Weisbach)

Considérons un écoulement du point A vers le point B. La conservation de la charge totale du fluide implique (en utilisant l'équation de Bernoulli) :

$$\frac{P_A}{\gamma} + \frac{V_A^2}{2g} + Z_A = \frac{P_B}{\gamma} + \frac{V_B^2}{2g} + Z_B + h_{fA \rightarrow B}$$



$$h_f = f \frac{L}{D} \frac{V^2}{2g} \text{ (m)}$$

$$\Delta p = f \frac{L}{D} \rho \frac{V^2}{2} \text{ (Pa)}$$

Où f est le coefficient de frottement

I-3) Coefficient de frottement

• Régime laminaire	$f = \frac{64}{Re}$
• Régime turbulent : résolution de l'équation de Colebrook-White	$\frac{1}{\sqrt{f}} = -2 \log_{10} \left(\frac{\epsilon/D}{3.7} + \frac{2.51}{Re\sqrt{f}} \right)$

I-4) Perte de charge singulières

$$h_s = K \frac{V^2}{2g} \text{ (m)}$$

$$\Delta p_s = K \rho \frac{V^2}{2} \text{ (Pa)}$$

avec K coefficient dépendant de la singularité

II- Manipulation : Réalisation du Diagramme de Moody

Objectif : Implémenter le diagramme de Moody sous MATLAB

Travail à réaliser :

II-1) Implémentation des fonctions de calcul

- Écrire une fonction $f_{lam}(Re)$ retournant $f = 64/Re$;
- Écrire une fonction $f_{turb}(Re, \epsilon/D)$ résolvant l'équation de Colebrook-White par méthode itérative (méthode du point fixe) ;
- Écrire une fonction principale $f_{darcy}(Re, \epsilon/D)$ sélectionnant automatiquement le régime.

II-2) Génération du diagramme

- Créer un vecteur de nombres de Reynolds de 10^2 à 10^8 (échelle logarithmique) ;
- Définir une liste de rugosités relatives typiques : $[0 \ 10^{-6} \ 10^{-5} \ 10^{-4} \ 5 \times 10^{-4} \ 10^{-3} \ 2 \times 10^{-3} \ 0,02]$;
- Calculer f pour chaque couple $(Re, \epsilon/D)$;
- Tracer le diagramme avec échelles log-log

II-3) Validation

- Vérifier la valeur $f = 0,316/Re^{0,25}$ pour les conduites lisses en régime turbulent
- Confronter les résultats avec des diagrammes de Moody de référence

III- Mini-Projets

III-1) Mini projet 1 : Calcul de Débit par Méthode Itérative

Énoncé du problème : De l'eau ($\rho = 1000 \text{ kg/m}^3$, $\mu = 10^{-3} \text{ Pa.s}$) s'écoule dans une conduite en fonte ($\epsilon = 0,26 \text{ mm}$) de diamètre $D = 15 \text{ cm}$ et de longueur $L = 100 \text{ m}$. La différence de pression mesurée est de $\Delta p = 50 \text{ kPa}$. La conduite comprend 3 coudes à 90° standard ($K = 0,9$ chacun) et une vanne entièrement ouverte $K = 0,2$.

Travail demandé :

1. Écrire un algorithme itératif pour calculer le débit Q
2. Implémenter la méthode sous MATLAB
3. Analyser la convergence de l'algorithme
4. Valider les résultats par vérification a posteriori

Méthodologie : Utilisation de la fonction f_{darcy} développée au paragraphe précédent dans une boucle itérative convergeant vers la solution.

III-2) Mini Projet 2 : Dimensionnement de Conduite

Énoncé du problème: vous devez transporter $Q = 10 \text{ L/s}$ d'eau sur une longueur $L = 200 \text{ m}$ avec une perte de charge totale ne devant pas excéder $\Delta p_{max} = 20 \text{ kPa}$. La conduite est en PVC lisse ($\epsilon \approx 0$).

Travail demandé :

1. Déterminer le diamètre minimal nécessaire parmi les diamètres commerciaux disponibles :
50, 63, 75, 90, 110 mm
2. Développer un programme MATLAB automatisant cette sélection
3. Analyser l'influence du diamètre sur les pertes de charge
4. Proposer une solution technique justifiée

Approche : Calcul pour chaque diamètre commercial et sélection du plus petit diamètre satisfaisant la contrainte $\Delta p \leq \Delta p_{max}$.

AÉRODYNAMIQUE - ÉTUDE COMPARATIVE

DES PROFILS NACA 2412 ET NACA 0012

But

Ce TP a pour objectif d'étudier et de comparer les caractéristiques aérodynamiques de deux profils d'aile distincts : le NACA 2412 (profil cambré) et le NACA 0012 (profil symétrique). À travers des simulations numériques sous MATLAB, nous analyserons l'influence de la cambrure sur les coefficients de portance et de traînée, établirons les polaires aérodynamiques de chaque profil et déterminerons leurs performances respectives en termes de finesse. Cette étude permettra de comprendre les compromis conceptionnels dans le choix d'un profil selon l'application aéronautique visée.

Objectifs pédagogiques :

- Implémenter les méthodes de calcul des coefficients aérodynamiques
- Visualiser l'effet de la cambrure et de l'incidence sur la performance
- Analyser le compromis portance/traînée pour différents profils

I- Partie théorique

I-1) Description des profils NACA

La NACA (National Advisory Committee for Aeronautics), prédécesseur de la NASA, a développé dans les années 1930-1950 une série de profils aérodynamiques standardisés. La famille NACA à 4 chiffres, utilisée dans ce TP, se décrit ainsi :

- **Premier chiffre** : Cambrure maximale en % de la corde
- **Deuxième chiffre** : Position de la cambrure maximale en dizaines de % de la corde
- **Derniers chiffres** : Épaisseur maximale en % de la corde

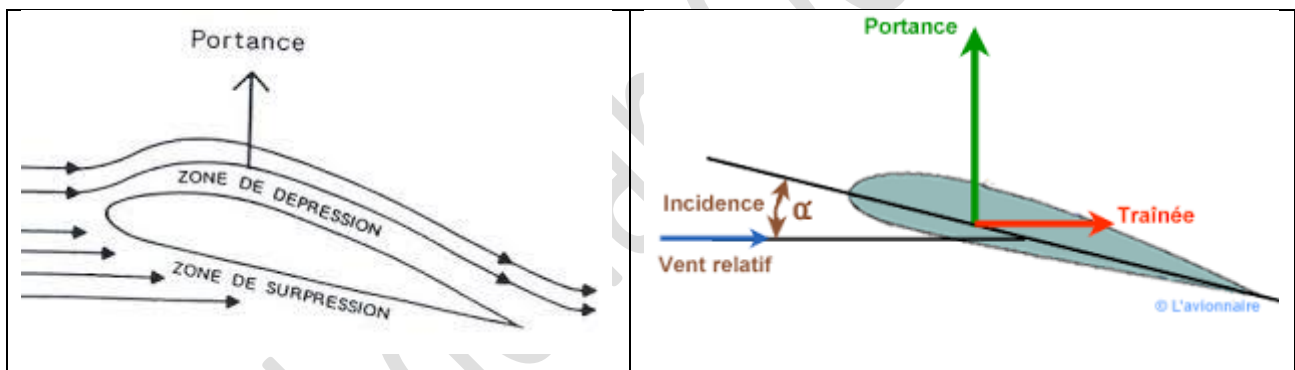
Ainsi :

- **NACA 2412** : 2% de cambrure à 40% de la corde, 12% d'épaisseur
- **NACA 0012** : 0% de cambrure, 12% d'épaisseur (profil symétrique)

I-2) Théorie de l'aile

Une aile dans un écoulement peut être étudiée en 2D. La théorie des profils mince (voir l'article sur [Théorie des profils minces](#)) permet d'expliquer l'apparition de la force portance sur un profil mince

dans un écoulement en utilisant une théorie de fluide parfait avec circulation de vitesse. La forme du profil permet en effet de **créer une circulation de vitesse autour du profil** (dans le sens des aiguilles d'une montre pour le schéma de droite). Cette circulation va accélérer la vitesse sur l'extrados (partie supérieure de l'aile) et au contraire diminuer la vitesse sur l'intrados (partie inférieure). En utilisant Bernoulli, on a donc une dépression sur l'extrados et une surpression sur l'intrados, ce qui génère une force de portance (noté L pour lift sur le schéma) perpendiculaire à la direction de la vitesse incidente (donc ici dans la direction verticale). La répartition de pression engendre aussi une force de traînée dans la direction horizontale, dite traînée de forme. Alors que la force de portance est bien prédite par la théorie fluide parfait des profils minces, la force de traînée (noté D pour drag sur le schéma) est-elle essentiellement d'origine visqueuse. La théorie précédente n'est donc pas suffisante pour la prédire. Il faut utiliser la théorie de la couche limite pour espérer la prédire correctement. Lorsque l'angle d'incidence augmente et dépasse une valeur limite, la couche limite sur l'extrados décolle et le profil décroche avec une apparition de tourbillon et une perte soudaine de portance (voir image ci-dessous de l'expérience de Prandtl). On limitera donc l'étude à des angles d'incidence compris entre -5° et $+15^\circ$.



I-3) Théorie de la portance aérodynamique

La portance (L) et la traînée (D) s'expriment par :

$$L = \frac{1}{2} \rho V^2 S C_L$$

$$D = \frac{1}{2} \rho V^2 S C_D$$

où :

- ρ : masse volumique de l'air ;
- V : vitesse de l'écoulement ;
- S : surface de référence ;
- C_L : coefficient de portance adimensionnel ;
- C_D : coefficient de traînée adimensionnel.

I-4) Effet de la cambrure

La cambrure modifie fondamentalement le comportement aérodynamique :

- **Profil cambré (NACA 2412)** : Génère une portance même à incidence nulle grâce à la dissymétrie de l'écoulement. La ligne moyenne courbée crée une différence de longueur de parcours entre l'extrados et l'intrados.
- **Profil symétrique (NACA 0012)** : Portance nulle à incidence nulle. Comportement identique en positif et négatif, idéal pour les applications nécessitant une symétrie des performances.

I-5) La polaire aérodynamique

La polaire (C_L en fonction de C_D) représente l'efficacité aérodynamique du profil. La pente de la droite issue de l'origine à la polaire donne la finesse (rapport portance/trainée) à une incidence donnée.

I-6) Equations à implémenter

- Le modèle utilise une **approximation linéaire** pour la portance dans la plage normale de vol

La **trainée** suit un **modèle polynomial** classique : $C_D = C_{D0} + k C_L^2$

- La **correction aux forts angles** capture l'augmentation brutale de trainée au décrochage
- Les **différences profil cambré/symétrique** sont modélisées via les paramètres C_{L0} et $C_{D0-base}$.

1. Classification du profil

Cambrure moyenne :

$$cambrure_moyenne = (1/n) \times \sum_{i=1}^n |y_{u_i} - y_{l_i}|$$

Critère :

- Si $cambrure_moyenne > 0,01 \rightarrow$ Profil cambré (NACA 2412)
- Sinon $\rightarrow$ Profil symétrique (NACA 0012)

2. Coefficient de portance C_L

$$C_L = C_{L0} + \frac{dC_L}{d\alpha} \alpha$$

Paramètres spécifiques :

Profil	C_{L0}	$\frac{dC_L}{d\alpha}$
NACA 2412	0,2	0,095
NACA 0012	0,0	0,1

3. Coefficient de trainée C_D

Coefficient de frottement :

$$C_f = \begin{cases} \frac{0.06}{\sqrt{Re}} & \text{si } Re < 5 \times 10^5 \\ \frac{0.074}{Re^{0.2}} & \text{si non} \end{cases}$$

Traînée de profil :

$$C_{D0} = 1.1 \times C_f + C_{D0-base}$$

- NACA 2412 : $C_{D0-base} = 0,008$

- NACA 0012 : $C_{D0-base} = 0,006$

Traînée induite :

$$C_{Di} = \frac{C_L^2}{\pi \times AR \times e}$$

- $AR = 8,0$ (allongement)

- $e = 0,9$ (facteur d'Oswald)

Traînée totale :

$$C_D = C_{D0} + C_{Di}$$

4. Correction pour forts angles d'attaque

Pour prendre en compte le phénomène de décrochage :

$$C_D + 5 \times 10^{-3}(|\alpha| - 12)^2 \text{ si } \alpha > 12$$

II- Manipulation

II-1) Fichier fourni

Le responsable du TP vous fournira le fichier [naca4.m](#), qui calcule la géométrie exacte du profil selon les équations standards NACA. La fonction détermine la ligne moyenne et y ajoute l'épaisseur pour générer l'extrados et l'intrados.

II-2) Fichiers à créer

[calcul_coefficients_corrige.m](#) : Implémente un modèle aérodynamique simplifié différenciant les comportements des profils cambrés et symétriques. Prend en compte la traînée de frottement, la traînée de forme et la traînée induite.

[TP_Aeorodynamique.m](#) Script principal orchestrant l'ensemble de l'étude comparative.

II-3) Déroulement de la manipulation

- **Génération des géométries** : Exécution de [naca4.m](#) pour les deux profils
- **Calcul des coefficients** : Pour une plage d'incidence de -5° à $+15^\circ$

- **Tracé des polaires** : Visualisation comparative $C_L = f(C_D)$
- **Analyse des performances** : Calcul des finesses maximales et incidences optimales

Question 1 : Observez-vous la différence de portance à incidence nulle entre les deux profils ? Quelle en est la cause physique ?

Question 2 : La pente de la courbe $C_L = f(\alpha)$ est-elle identique pour les deux profils ? Justifiez.

Question 3 : Quel profil offre la meilleure finesse maximale ? Dans quel contexte d'utilisation serait-il préférable ?

Question 4 : Comment évolue l'incidence de portance nulle entre les deux profils ?

Bonus1 : Effectuer une étude paramétrique de l'influence du nombre de Re ;

Bonus2 : performance pour des grandes valeurs de α

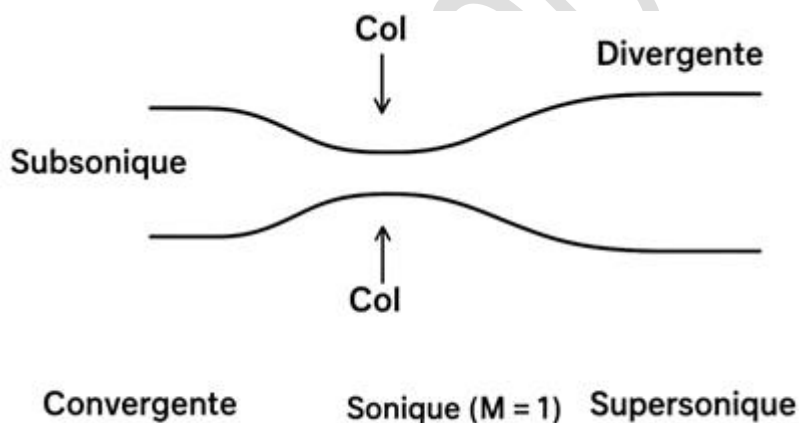
ÉTUDE DES ÉCOULEMENTS COMPRESSIBLES DANS UNE TUYÈRE DE LAVAL

I- But du TP

L'objectif de ce TP est d'étudier l'écoulement compressible stationnaire dans une tuyère de Laval :

- calculer le débit massique et les profils le long de la tuyère (Mach, pression, température) ;
- comprendre la relation *aire – Mach* (A/A^*) ;
- détecter et traiter la présence d'un choc normal dans la partie divergente ;
- implémenter un algorithme numérique (MATLAB) qui trace les profils et le débit massique normalisé.

II- Partie théorique



II-1) . Hypothèses

- Écoulement unidimensionnel le long de l'axe de la tuyère ;
- Écoulement parfait (gaz parfait), sans frottement et adiabatique (isentropique) sauf au choc ;
- Rapport des chaleurs spécifiques constant $\gamma = 1,4$ et constante des gaz $r = 287 \text{ J/kg/K}$.

II-2) . Relations isentropiques

Pour un écoulement isentropique entre un état stagnation ($P_0; T_0; \rho_0$) et un état local ($P; T; \rho; M$):

$$\frac{T}{T_0} = \left(1 + \frac{\gamma - 1}{2} M^2\right)^{-1}$$

$$\frac{P}{P_0} = \left(1 + \frac{\gamma - 1}{2} M^2\right)^{\frac{\gamma}{\gamma - 1}}$$

$$\frac{\rho}{\rho_0} = \left(1 + \frac{\gamma - 1}{2} M^2\right)^{\frac{1}{\gamma - 1}}$$

La vitesse locale V s'obtient par :

$$V = M \sqrt{\gamma r T}$$

et le débit massique local

$$\dot{m}(x) = \rho(x) V(x) A(x)$$

II-3) . Relation aire - Mach (A/A^*)

La relation liant l'aire réduite (A/A^*) au nombre de Mach est (pour un écoulement isentropique 1D)

$$\frac{A}{A^*} = \frac{1}{M} \left[\frac{2}{\gamma + 1} \left(1 + \frac{\gamma - 1}{2} M^2 \right) \right]^{\frac{\gamma + 1}{2(\gamma - 1)}}$$

Cette équation est non linéaire en (M) et doit être résolue numériquement (deux branches : sous-critique ($M < 1$) et sur-critique ($M > 1$)).

II-4) . Choc normal (relations de saut)

Pour un choc normal, si ($M_1 > 1$) est le Mach en amont du choc et ($M_2 < 1$) le Mach en aval :

$$M_2 = \sqrt{\frac{1 + \frac{\gamma - 1}{2} M_1^2}{\gamma M_1^2 - \frac{\gamma - 1}{2}}}$$

La relation de pression totale (pression statique) traversant le choc est :

$$\frac{P_2}{P_1} = 1 + \frac{2\gamma}{\gamma + 1} (M_1^2 - 1)$$

La densité traverse selon la relation :

$$\frac{\rho_2}{\rho_1} = \frac{(\gamma + 1) M_1^2}{(\gamma - 1) M_1^2 + 2}$$

II-5) . Algorithme de calcul (pseudo-code)

1. Définir paramètres physiques ($P_0; T_0; \rho_0$) et le profil d'aire $A(x)$. Trouver $A^* = \min(A)$;
2. Calculer l'état critique ($M^* = 1$) : ρ^*, p^*, T^*, V^* ;
3. Pour chaque pression de sortie P_{out} de la plage demandée :
 - Pour chaque position x_i résoudre la relation $A(x_i)/A^* = F(M)$ pour obtenir $M(x_i)$; Choisir la branche (sous/sur) en fonction de la position (avant sommet : subsonique ; après sommet : supersonique) ou d'une condition initiale ;

- Calculer $P(x), \rho(x), T(x), V(x)$ par relations isentropiques ;
 - Vérifier si la pression aval imposée P_{out} impose l'apparition d'un choc normal : comparer P_{out} avec la pression isentropique en sortie. Si P_{out} est suffisamment grande, rechercher le premier point en partant de la sortie où un choc s'imposerait. Appliquer les relations de choc pour modifier (M, P, ρ, T, V) en aval du point de choc ;
 - Calculer le débit local $\dot{m}(x)$ et normaliser par $\dot{m}^*$;
4. Tracer les courbes $\dot{m}/\dot{m}^*$: le long de la tuyère et les profils (M, P, T) .

III- . Partie programmation (définition des étapes du code)

1. **Initialisation** : effacer l'espace de travail (`clear; clc; close all;`), définir constantes physiques, conditions stagnantes, discrétisation spatiale et profil d'aire $A(x)$;
2. **Calculs préliminaires** : déterminer A^* , *calculer les variables au point critique $M = 1$* ;
3. **Boucle principale sur P_{out}** : pour chaque pression de sortie choisie, calculer le profil de Mach en résolvant numériquement l'équation $A/A^* = F(M)$ en chaque point. Utiliser `fsolve` ou une méthode de Newton-Raphson avec des conditions initiales adaptées (branche subsonique en amont, supersonique en aval) ;
4. **Calcul isentropique** : à partir de $M(x)$ calculer $P(x), \rho(x), T(x), V(x)$;
5. **Détection et traitement du choc** : si la condition sur P_{out} nécessite un choc, trouver l'indice du choc en partant de la sortie et appliquer les relations de saut pour M, P et ρ ; recalculer T et V en aval ;
6. **Calcul du débit massique et tracé** : calculer $\dot{m}(x)$, normaliser par $\dot{m}^*$ et tracer les courbes (débit normalisé, profils M, P, T , profil $A(x)$) ;
7. **Options d'affichage** : personnaliser légendes, marquer la position du choc sur les figures.

III-1) Points de vigilance en programmation

- Le solveur non-linéaire doit être contrôlé (*initial guess* selon la région) pour éviter de converger sur la branche indésirable.
- Vérifier les dimensions et unités ($Pa, K, m, kg, etc.$).
- Traiter le cas sans choc et le cas avec choc.

IV- 4. Partie Questions

1. Expliquez pourquoi la relation A/A^* admet deux solutions (subsonique et supersonique) ;
2. Comment la pression de sortie impose-t-elle l'apparition d'un choc dans la partie divergent? Donnez un raisonnement énergétique et mécanique ;
3. Montrez numériquement l'effet d'une modification du profil $A(x)$ sur le débit massique maximal ;
4. Pourquoi normalise-t-on le débit par $\dot{m}^*$?
5. Proposez (sans la programmer) une amélioration numérique pour localiser de façon plus robuste la position du choc (méthode numérique alternative) ;
6. Pouvez modifier cette valeur et le profil $A(x)$ selon vos besoins expérimentaux.

V- 5. Conclusion

Ce TP permet d'appliquer les lois de la thermodynamique et de la mécanique des fluides compressibles pour comprendre le rôle du profil de tuyère et des conditions aux limites (pression de sortie) sur la formation d'un choc et le débit massique. L'implémentation MATLAB fournie sert de base pour des études paramétriques et des optimisations du profil.